A NoSQL Report on

Scheme Eligibility Assistant: Explainable Government-Scheme Eligibility & Grounded Q&A on MongoDB

Submitted to Manipal University Jaipur

Towards the partial fulfillment for the Award of the Degree of

B. Tech Computer Science and Engineering (Artificial Intelligence and Machine Learning)

In Computer Science and Engineering (AIML)

2024-2028

By

Nalin Ahuja (2427010036)

Adamya Singh Thakur (2427010181)



Under the guidance of

Ms. Surabhi Agrawal

Department of Artificial Intelligence and Machine Learning

Manipal University Jaipur

Jaipur, Rajasthan

Abstract

Every year the central and state governments announce a large number of welfare schemes, yet many people who could benefit from them never apply. In most cases they simply do not know that they qualify. The rules are written in long pages of official text, each scheme phrases them differently, and they keep getting revised. We built the **Scheme Eligibility Assistant** to close this gap. It is a web application that works in two ways. First, a user fills in a short profile and our rules engine goes through every published scheme and tells them whether they are eligible, not eligible, or whether we still need some detail from them — and for each scheme it shows which condition decided the answer. Second, the user can type a question in plain English and get an answer that is put together only from the scheme text we have stored, with a link back to the source. If our data does not cover the question, the assistant says so instead of making something up. Signed-in citizens also get a ranked list of recommended schemes with the reasons behind each rank, an application tracker with a document checklist and deadline reminders, and a Grievance Portal for complaints. Citizens and officers sign in on separate tabs of a login page, and officers get their own dashboard, which includes a coverage-gap analysis that maps where people search for help that the scheme data does not cover.

We used MongoDB for all of the storage. A scheme and its eligibility conditions live together in one document. Whenever the conditions change we save a fresh version instead of editing the old one, and this save happens inside a transaction. Every check a user runs is recorded with a copy of the exact rules and answers that were used, plus a hash so that tampering can be detected later. For question answering we combine MongoDB text search with similarity search over stored embeddings. Our data comes from 601 scheme pages published on myScheme (Hugging Face dataset shrijayan/gov_myscheme), which we cleaned into 570 schemes — 113 central and 457 from 34 states and union territories. One idea guided the whole design: eligibility is always decided by plain, testable code, never by a language model.

Keywords: MongoDB, document database, schema validation, aggregation, $facet, $bucket, geospatial index, transactions, TTL index, full-text and vector search, rules engine, recommendation ranking.

Contents

1. Problem identification and real-world relevance
2. Objectives and scope
3. NoSQL database selection and justification
4. System architecture
5. Database design and data modelling
6. CRUD implementation
7. Advanced NoSQL features and query optimisation
8. Rules engine design
9. Grounded retrieval-augmented Q&A
10. Application integration and functionality
11. Testing, security and data validation
12. Results
13. Limitations and future work
14. Conclusion
15. References
Appendix A — Setup and run instructions
Appendix B — Plagiarism report

1. Problem identification and real-world relevance

1.1 Problem statement

Suppose a farmer in Assam wants to know which government schemes she can apply for. Today she would have to open scheme pages one by one and work out conditions such as age limits, income ceilings, caste category, land owned, BPL status and state of residence — and then do the same for the next scheme, and the next. Most people do not have the time for this, so they either give up or pay an agent. Staff at Common Service Centres and block offices run into the same problem, only with many more applicants. A general chatbot is not a good answer either: if it confidently tells someone the wrong thing about a benefit, that person can lose money they were entitled to.

1.2 Who it serves

- **Citizens**, who want to find out what they can claim without handing over identity documents.
- **CSC operators and block officers**, who screen many applicants a day and need answers they can explain.
- **Scheme administrators**, who need to change rules safely and look back at decisions made earlier.

1.3 Why it matters

- A lot of welfare money goes unclaimed because the people it is meant for never hear that they are eligible.
- A useful answer has to be **explainable** — the person should see which condition they met or missed — and **reproducible**, meaning we can show what the rules were on the day they checked.
- Scheme rules are revised from time to time. Showing when our data was last verified, and keeping old rule versions instead of overwriting them, are practical requirements for a system like this.

2. Objectives and scope

2.1 Objectives

- Store scheme data whose structure varies (central and state schemes are not laid out the same way) in a document database.
- Write eligibility conditions as data — a tree of rules — so that a single piece of code can check all the schemes, instead of writing separate code for each one.
- Give one of three answers for every scheme (Eligible, Not eligible, Need more information), explain each condition, and suggest the one question that would help the user most.
- Keep a record of every check that cannot be quietly edited and that can be re-run later from what it stored.
- Never edit published rules in place; publish each change as a new version in a single atomic step.
- Answer free-text questions using only stored scheme text, show the sources, and refuse when the sources do not contain the answer.
- Give users a way to raise a complaint about an application or a result, track it by ticket number, and see every update from the officer handling it.
- Rank the schemes that fit a person and explain the rank; let citizens save schemes and track their own applications; show officers where demand for schemes is not being met.

- Cover the database topics of the course: CRUD, indexing, aggregation, validation, TTL, text and vector search, login and roles, and automated tests.

2.2 Scope

Included in this project	Left for future work
570 schemes from the myScheme dataset; rules for 16 of them checked by hand	Live updates from the official API Setu / myScheme services
Eligibility check, ranked recommendations, Q&A, scheme browser, tracker, grievances; officer dashboard	Applying to a scheme on the user's behalf
English text (a multilingual embedding model can be swapped in later)	A full Hindi or regional-language interface
MongoDB running locally as a replica set; the code also supports Atlas	Cloud hosting and production monitoring

Note: this is a college project built on public data. It is not an official government service. Every scheme in the app links to its page on myScheme so that users can confirm the details there.

3. NoSQL database selection and justification

3.1 Why a document database

When we looked at the data, a scheme turned out to be one nested object: a few text sections, a list of benefits, a list of documents to submit, and an eligibility condition that can be nested to any depth — for example A AND (B OR C) AND NOT D. Central and state schemes do not fit the same set of columns. If we had used a relational database, we would have needed separate tables for schemes, conditions, condition groups, benefits and documents, and joined them back together on every request. The nested condition tree would still have needed either a self-referencing table queried recursively or a JSON column. A document database lets us keep the whole thing together and read it in one go.

What we needed	With a relational database	With MongoDB (our choice)
Schemes with different fields	Many empty columns, or key-value tables	Flexible documents, checked by a JSON Schema validator
Nested eligibility conditions	Self-referencing table with recursive queries, or JSONB	Stored as a nested sub-document and read in one fetch
Load a scheme with its rules	Several joins	One document read
Audit record with the exact inputs	Joins back to versioned tables that must never change	Copies embedded in the record itself
Text plus similarity search	Extra extensions	Text index and vector search next to the data
Publish rules atomically	Transactions	Multi-document transactions on a replica set
Growing audit log	Partitioning and read replicas	Sharding on user or run keys; replica sets

3.2 Why MongoDB in particular

- **Validators on collections.** We keep the flexibility of documents, but the server still refuses bad data such as an unknown rule operator, an invalid verdict, a negative age or a missing checksum.
- **Aggregation pipelines.** Reports such as how many checks ended in each verdict, or which answers were missing most often, are computed inside the database.
- **Transactions.** On a replica set we can save a new rule version and point the scheme to it as one step.
- **TTL indexes.** Chat logs and rate-limit counters delete themselves after a set time, so we do not keep data longer than needed.
- **Search.** A text index handles keyword matching. On Atlas the code uses $vectorSearch for similarity; on our local server it computes the same similarity from embeddings stored in MongoDB.
- **A good Python driver.** PyMongo works well with our FastAPI backend.

3.3 What we gave up

MongoDB does not have foreign keys, so we had to protect the data in other ways. We added unique indexes (scheme slug, slug plus version, email, one profile per user), validators on the important collections, Pydantic checks on every API request, and our own rule validator that runs before anything is published. We also follow a simple rule in the code: published versions and audit records are only ever inserted, never updated.

4. System architecture

The system has three parts that all use the same MongoDB database. The ingestion part reads the scheme PDFs and prepares the data offline. The rules engine decides eligibility. The Q&A part finds relevant text and writes answers, and whenever a question is really about eligibility it passes the work to the rules engine. The recommendation ranking, the application tracker, the grievance portal and the coverage analytics use the same database. The browser only ever talks to our FastAPI server, through three pages: the public site, the login page and the officers' dashboard.

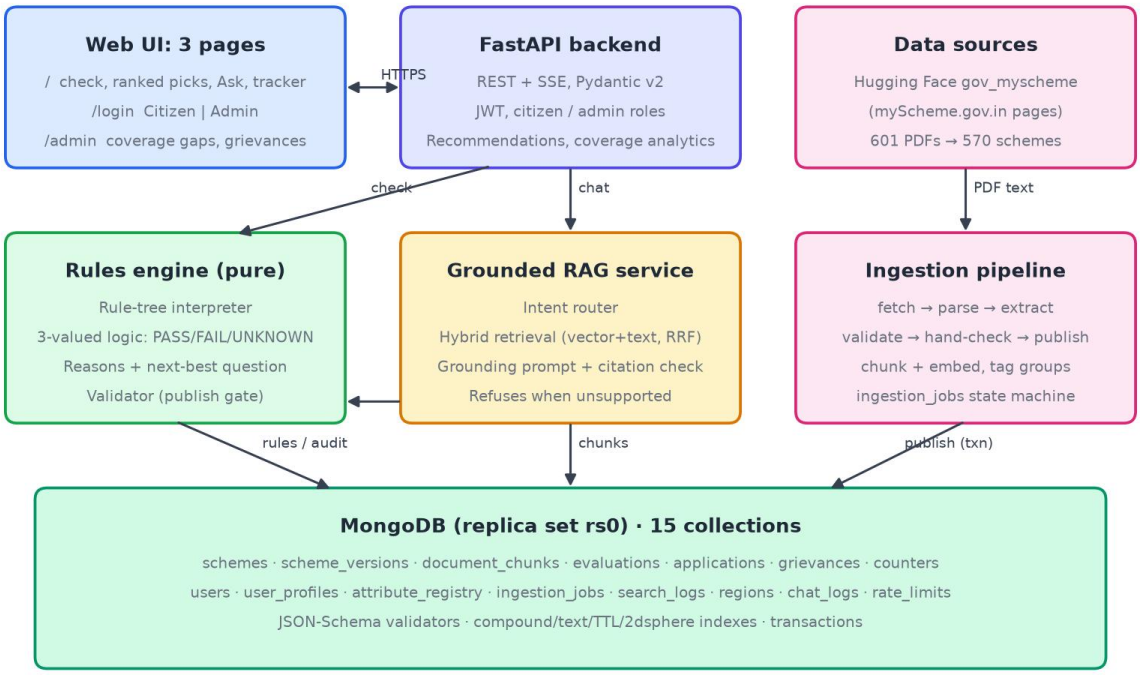


Figure 1 — System architecture

Layer	What we used	Why
Frontend	Three pages (public site, /login, /admin) in HTML and plain JavaScript; Leaflet for the map	No build step, supports streamed answers; the map needs internet for tiles
Backend	Python 3.12, FastAPI, Pydantic v2, Uvicorn	Typed endpoints and automatic API documentation at /docs
Database	MongoDB 8.3 as a one-node replica set (rs0)	Transactions need a replica set; the same code runs on Atlas
Driver	PyMongo 4	Official driver with sessions, transactions and aggregation
Embeddings	TF-IDF followed by TruncatedSVD (LSA), 256 dimensions	Works without downloading a model; can be replaced later
Language model (optional)	Claude through the Anthropic SDK	Only rewrites retrieved text; without a key we build the answer from the text itself
Login	JWT tokens and PBKDF2-SHA256 password hashes	Separate Citizen and Admin sign-in; the server checks the role
Testing	pytest, Hypothesis, FastAPI TestClient	Logic tables, random property tests, sample profiles and API tests

5. Database design and data modelling

We followed one rule of thumb while designing the collections: **keep together what is read together, keep separate what changes on its own, and copy anything that must never change.** In total the database has fifteen collections.

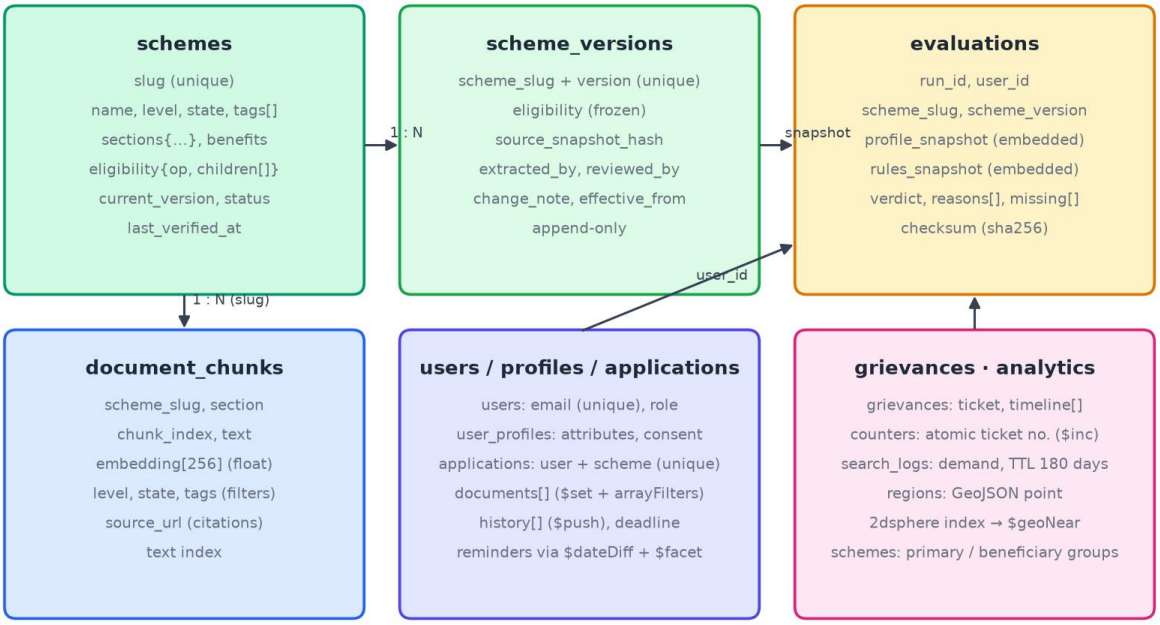


Figure 2 — Collections and how they relate

Collection	What it stores	Pattern	Main indexes
schemes	One scheme with its text sections, tags and current rules	Embedded rule tree	slug (unique); {status, level, state}; tags; text index
scheme_versions	A frozen copy of each published rule set	Insert-only, versioned	{scheme_slug, version} unique
evaluations	One record per scheme per check, with copies of the inputs and a checksum	Embedded copies, insert-only	{user_id, evaluated_at}; run_id; {scheme_slug, scheme_version}
document_chunks	Pieces of scheme text with their 256-number embeddings	Linked by slug; filter fields copied in	{scheme_slug, chunk_index} unique; {level, state}; text index
users	Email, password hash and role	Plain	email unique
user_profiles	A user's saved answers and consent	Flexible sub-document	user_id unique
attribute_registry	The list of questions a rule is allowed to use	Reference data	key unique
ingestion_jobs	Where each scheme is in the pipeline, and its proposed rules	Status field	{scheme_slug, source_hash} unique; {stage, updated_at}
chat_logs	Questions asked and the sources used	Expires	TTL on created_at (30 days)
rate_limits	Request counts per IP address	Expires	TTL on expires_at; {key, window} unique
grievances	One complaint with its ticket, status, category and a timeline of every update	Embedded timeline, grows with $push	ticket unique; {user_id, created_at}; {status, updated_at}; scheme_slug
counters	The last ticket number used each year	Single counter document	_id per year
applications	A citizen's saved scheme: own status, deadline, document checklist, status history	Embedded arrays	{user_id, scheme_slug} unique; {user_id, deadline}
search_logs	Each search: topic words, groups, state, number of matching schemes	Expires after 180 days	{state, created_at}; q_norm; TTL on expires_at

regions	Each state / UT with a GeoJSON point	Reference data	2dsphere on location; name unique

5.1 A scheme document

Below is a shortened version of the stored document for Atal Pension Yojana. The rules sit inside the document, and each condition remembers the sentence it was taken from.

```
{ "slug": "apy", "name": "Atal Pension Yojana", "level": "central", "state": null,
  "status": "published", "current_version": 1,
  "source_url": "https://www.myscheme.gov.in/schemes/apy",
  "eligibility": { "op": "AND", "children": [
      { "id": "r1", "attribute": "age", "operator": "between", "value": [18, 40],
        "source_quote": "The minimum age of joining APY is 18 years and maximum is 40 years." },
      { "op": "NOT", "children": [
          { "id": "r2", "attribute": "is_income_tax_payer", "operator": "==", "value":
true } ] } ] },
  "unmapped_conditions": ["Contribution by auto-debit from a savings bank account"] }
```

A condition can compare values (==, !=, <, <=, >, >=), check membership (in, not_in), check a range (between) or check that a value is present (exists). Conditions are combined with AND, OR and NOT, and these groups can be nested. Anything we could not turn into a condition is listed under unmapped_conditions and shown to the user as a reminder to check it themselves.

5.2 An evaluation (audit) record

```
{ "run_id": "run_7f3c...", "scheme_slug": "apy", "scheme_version": 1,
  "evaluated_at": ISODate("2026-10-21T08:12:44.120Z"),
  "profile_snapshot": { "age": 29, "is_income_tax_payer": false },
  "rules_snapshot": { ...copy of version 1 of the rules... },
  "verdict": "ELIGIBLE", "reasons": [ ... ], "missing_attributes": [],
  "checksum": "sha256:9a1e..." }
```

Since each record carries its own copy of the rules and answers it used, we can re-run it later without looking anything up, and it stays correct even after the scheme or the user's profile changes. We only copy the answers that the scheme's rules actually need, so we do not store more personal data than necessary.

5.3 How the design serves each query

What the app needs to do	How the data model handles it
Check a profile against every scheme	The {status, level, state} index picks published central schemes plus those of the user's state; each scheme is one read
Show a scheme's page	One document fetched by its unique slug
Show a user's past checks, newest first	The {user_id, evaluated_at} index returns them already sorted
Show every result of one check	Index on run_id
Find checks that used a given rule version	Index on {scheme_slug, scheme_version}
Search by scheme name	Text index on chunks, with the scheme name weighted five times higher
Search by meaning	A 256-number vector on each chunk; $vectorSearch on Atlas, cosine similarity locally

5.4 A grievance document

A complaint and its whole history are read together, so the history is stored inside the complaint as an array. Each reply from the user or the officer is one more entry in that array.

```
{ "ticket": "GRV-2026-000001", "user_id": ObjectId("..."), "scheme_slug": "apy",
  "category": "application_status", "subject": "Application not processed",
  "status": "resolved", "priority": "normal", "run_id": "run_ab07...",
  "timeline": [
    { "at": ISODate("..."), "by": "user",  "action": "filed",         "status": "open" },
    { "at": ISODate("..."), "by": "admin", "action": "status_change", "status": "resolved",
      "note": "Bank confirmed your enrolment; your PRAN has been issued." } ] }
```

The optional run_id links the complaint to one of the user's own eligibility checks, so the officer can open the exact audit record the user is talking about.

5.5 Planning for growth

- The evaluations collection grows fastest, since every check adds one record per scheme. Because records are only inserted, it can be sharded on {user_id, evaluated_at} or on a hashed run_id, and old records can be moved to cheaper storage.
- Each text chunk carries copies of the scheme's level, state and tags so searches can filter on them directly.
- A scheme document is only tens of kilobytes, nowhere near MongoDB's 16 MB limit.
- Most traffic is reading schemes, which secondary members of the replica set can serve.

6. CRUD implementation

All database work is done with PyMongo inside the FastAPI route handlers. Data is checked twice before it is stored: once by Pydantic when the request arrives, and again by the collection's validator inside MongoDB.

Operation	Collection	Endpoint	MongoDB call
Create	users	POST /api/auth/register	insert_one; the unique email index blocks duplicates
Create	evaluations	POST /api/eligibility/check	insert_many, one record per scheme with a shared run_id
Create	scheme_versions	scripts/seed.py (hand-checked rules)	insert_one inside a transaction
Create	ingestion_jobs	scripts/seed.py	update_one with upsert, keyed on slug and source hash
Read	schemes	GET /api/schemes	find with filters, text search and paging
Read	schemes	GET /api/schemes/{slug}	find_one by slug
Read	evaluations	GET /api/eligibility/runs/{run_id}	find by run_id
Read	evaluations	GET /api/admin/stats	aggregation pipelines
Update	user_profiles	PUT /api/profile	update_one with $set and upsert
Update	schemes	publish / retire	update_one inside the publish transaction
Delete	user_profiles, evaluations	DELETE /api/profile	delete_one and delete_many (user asks to erase their data)
Delete (soft)	schemes	POST /api/admin/schemes/{slug}/retire	status set to retired so history is kept

Create	grievances	POST /api/grievances	insert_one, with a ticket number from an atomic counter
Read	grievances	GET /api/grievances · /api/admin/grievances	find by user_id; aggregation with $lookup for officers
Update	grievances	POST .../comments · .../status	update_one with $push to the timeline and $set on status
Create	applications	POST /api/applications	insert_one; the unique {user_id, scheme_slug} index blocks a second save
Update	applications	PATCH /api/applications/{slug}[/documents/{id}]	$set with arrayFilters on one checklist item; $push to history
Delete	applications	DELETE .../documents/{id} · DELETE /api/applications/{slug}	$pull one checklist item; delete_one to unsave

6.1 Creating a user

```
try:
    uid = db.users.insert_one({"email": body.email.lower(), "name": body.name,
                               "password_hash": hash_password(body.password),
                               "role": "user", "created_at": now()}).inserted_id
except DuplicateKeyError:
    raise HTTPException(409, "Email already registered")
```

6.2 Saving a profile (only with consent)

```
db.user_profiles.update_one({"user_id": user["_id"]},
    {"$set": {"attributes": clean, "consent": {"store_profile": True, "given_at": now()}}},
    upsert=True)
```

6.3 Deleting a user's data

```
db.user_profiles.delete_one({"user_id": user["_id"]})
db.evaluations.delete_many({"user_id": user["_id"]})
```

6.4 Adding to a grievance's timeline

```
db.grievances.update_one({"ticket": ticket}, {
    "$set":  {"status": "resolved", "updated_at": now(), "resolved_at": now()},
    "$push": {"timeline": {"at": now(), "by": "admin", "action": "status_change",
                           "status": "resolved", "note": note}}})
```

One call changes the status and adds the history entry together, so the timeline always agrees with the current status.

Our API tests confirm that the data is really saved: they create, read, change and delete records through the HTTP endpoints and then look in MongoDB directly to make sure the change happened.

7. Advanced NoSQL features and query optimisation

7.1 Validation inside the database

Each important collection has a $jsonSchema validator set to strict mode. For example, the evaluations collection refuses any record whose verdict is not one of our three values or whose checksum does not start with "sha256:". The schemes collection limits level, status and rule-group values and checks the slug format; user_profiles rejects ages outside 0–120 and negative income or land; users checks the

email format and role. During testing we tried inserting a record with the verdict "MAYBE", and MongoDB rejected it.

7.2 Publishing rules inside a transaction

```
with client.start_session() as s:
    s.with_transaction(lambda sess: (
        db.scheme_versions.insert_one(new_version, session=sess),
        db.schemes.update_one({"slug": slug},
            {"$set": {"eligibility": tree, "current_version": n + 1}}, session=sess)))
```

Both writes succeed together or not at all, so a scheme can never point to a version that was not saved. Each new version also records who checked it and a short note on what changed.

7.3 Aggregation pipelines

The admin dashboard is built from aggregation pipelines. Two of them are shown below. The first counts verdicts for each scheme. The second finds which missing answers held back the most results, which tells us which question to ask users first.

```
db.evaluations.aggregate([
  {"$group": {"_id": {"scheme": "$scheme_slug", "verdict": "$verdict"}, "n": {"$sum": 1}}},
  {"$group": {"_id": "$_id.scheme", "total": {"$sum": "$n"},
              "verdicts": {"$push": {"k": "$_id.verdict", "v": "$n"}}}},
  {"$sort": {"total": -1}}, {"$limit": 10}])

db.evaluations.aggregate([
  {"$match": {"verdict": "NEEDS_MORE_INFO"}}, {"$unwind": "$missing_attributes"},
  {"$group": {"_id": "$missing_attributes", "n": {"$sum": 1}}}, {"$sort": {"n": -1}}])
```

A third pipeline groups schemes by state and uses $cond to count how many in each state already have published rules.

7.4 Indexes and query performance

We checked our main queries with explain("executionStats"). To see what each index was worth, we ran the same query a second time with hint({$natural: 1}), which makes MongoDB scan the whole collection. The numbers below come from our own dataset after a load test.

Query	Full scan	Using the index	Plan
Check pre-filter: published, central or Assam	570 docs · 1 ms	17 docs · 0 ms	FETCH ← IXSCAN
User history, newest first	6506 docs · 7 ms	135 docs · 0 ms	FETCH ← IXSCAN
All verdicts of one run	6506 docs · 3 ms	15 docs · 0 ms	FETCH ← IXSCAN
Audits of one scheme version	6506 docs · 3 ms	431 docs · 1 ms	FETCH ← IXSCAN
Chunks of one scheme	3991 docs · 3 ms	9 docs · 0 ms	FETCH ← IXSCAN

- Compound indexes put equality fields first and the sort field after them, so a user's history comes back in order without a separate sort step.
- Large fields such as embeddings are left out of query results with projections.
- All the records from one check are written in a single insert_many call.
- Giving the scheme name a higher weight in the text index makes questions that name a scheme find it first.

7.5 TTL indexes

```
db.chat_logs.create_index("created_at", expireAfterSeconds=30 * 24 * 3600)
db.rate_limits.create_index("expires_at", expireAfterSeconds=0)
```

Chat questions are removed automatically after 30 days, and each rate-limit counter removes itself once its time window is over. Nothing in our own code has to clean these up.

7.6 Features used by the grievance portal

- **Atomic ticket numbers.** Each new complaint calls find_one_and_update on the counters collection with $inc and upsert. MongoDB does the increment in one step, so two complaints filed at the same moment can never get the same number. Our tests file two complaints in a row and check that the numbers follow on.
- **Embedded, growing timeline.** Replies are added with $push, so reading a complaint returns its full history in one fetch.
- **$lookup for the officer's view.** The admin queue uses an aggregation that joins each complaint with the filer's email from the users collection. The user's details stay in one place and are not copied into every complaint.
- **$facet for the dashboard.** One aggregation returns three results at once: complaints by status, complaints by category, and the average time to resolve (resolved_at minus created_at, in hours).
- **A validator on grievances.** The database accepts only our seven categories and four statuses, a ticket number in the GRV-YYYY-NNNNNN format, and a timeline with at least one entry.

```
db.counters.find_one_and_update({"_id": "grievance-2026"}, {"$inc": {"seq": 1}},
                                upsert=True, return_document=True)      # -> GRV-2026-000042

db.grievances.aggregate([{"$facet": {
    "by_status":   [{"$group": {"_id": "$status", "n": {"$sum": 1}}}],
    "by_category": [{"$group": {"_id": "$category", "n": {"$sum": 1}}}],
    "resolution":  [{"$match": {"resolved_at": {"$ne": None}}},
                    {"$group": {"_id": None, "avg_hours": {"$avg": {"$divide":
                        [{"$subtract": ["$resolved_at", "$created_at"]}, 3600000]}}}}]}}])
```

7.7 Combining text and vector search

For keyword matching we use MongoDB's $text operator and sort by its relevance score. For matching by meaning, every chunk stores a 256-number embedding. On Atlas the code sends a $vectorSearch query with level and state filters; on our local server it loads the embeddings from MongoDB once and computes cosine similarity with NumPy. We then merge the two ranked lists with Reciprocal Rank Fusion, where a chunk's score is the sum of 1 / (60 + its rank) over both lists.

7.8 Application tracker: updating inside embedded arrays

Each saved scheme is one applications document with the checklist and status history inside it. Ticking one document uses arrayFilters, so MongoDB changes exactly the matching item; adding uses $push and removing uses $pull. Reminders come from one aggregation that works out the days left with $dateDiff and splits the results with $facet. The status here is labelled as the citizen's own record, not the official government status.

```
db.applications.update_one({"user_id": uid, "scheme_slug": "aaby"},
    {"$set": {"documents.$[d].done": True}}, array_filters=[{"d.id": doc_id}])
db.applications.aggregate([{"$match": {"user_id": uid, "deadline": {"$ne": None}}},
    {"$addFields": {"days_left": {"$dateDiff": {"startDate": "$$NOW", "endDate": "$deadline",
"unit": "day"}}}},
```

```
    {"$facet": {"due_soon": [{"$match": {"$expr": {"$lte": ["$days_left", "$reminder_days"]}}}],
                "overdue":  [{"$match": {"days_left": {"$lt": 0}}}]}}])
```

7.9 Recommendation ranking with an aggregation pipeline

The rules engine first decides which schemes are possible; ineligible ones are never recommended. Schemes whose rules have not been hand-checked are pre-checked against their machine-read rules and shown with an "unverified" label. A pipeline then scores and sorts the candidates: 45% for the conditions met (worth less when a scheme's only rules are age, citizenship or state), 40% for how well the groups the scheme is meant for match the person's ($setIntersection), and 15% for how often users with a similar profile saved it ($lookup on applications). A scheme aimed only at someone else, such as a student scholarship for a 67-year-old, is skipped. Every result lists the parts of its score in plain words.

7.10 Coverage gap analytics: $facet, $bucket and geospatial queries

Supply is the schemes tagged with each beneficiary group; demand is the searches people make, logged with their state and how many schemes matched. One $facet stage computes supply by state and group, another computes demand. $bucket sorts states into bands by number of schemes, and searches into bands by number of matches. Each state is stored as a GeoJSON point with a 2dsphere index, so the map can ask for nearby states with $geoNear. The dataset has no district-level schemes, so regions are states and union territories, and for the demo 900 searches were generated by a script and clearly flagged as simulated.

```
db.regions.aggregate([{"$geoNear": {"near": bihar["location"], "distanceField": "dist_m",
                                    "maxDistance": 600_000, "spherical": True}}])
db.schemes.aggregate([{"$group": {"_id": "$state", "n": {"$sum": 1}}},
    {"$bucket": {"groupBy": "$n", "boundaries": [1, 5, 10, 20, 40], "default": "40+"}}])
```

8. Rules engine design

The rules engine (api/engine/evaluate.py) is a small recursive function that walks through a scheme's rule tree. It does not touch the database, which makes it easy to test and fast enough to check every scheme in a few milliseconds.

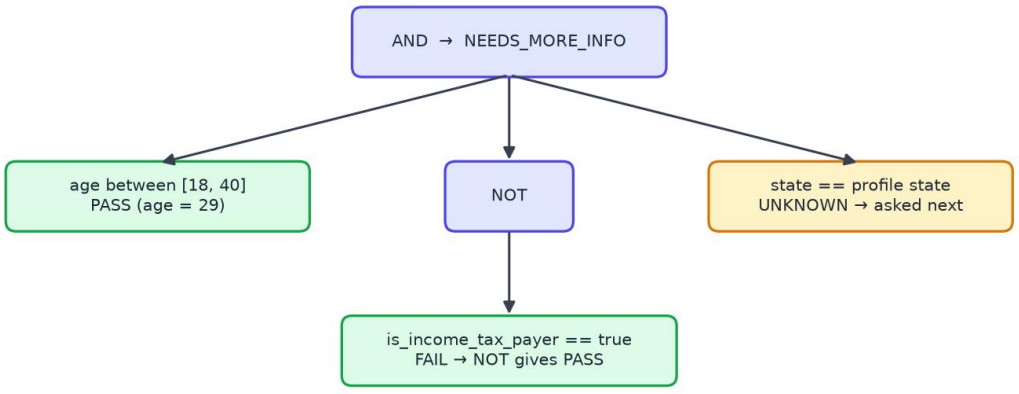


Figure 3 — Checking a rule tree with three possible results

8.1 Three possible results

If a user leaves a question blank, the condition that needs it becomes UNKNOWN rather than FAIL. So instead of wrongly saying "you are not eligible", the app says "tell us how much land you own and we can check".

AND	PASS	FAIL	UNKNOWN
PASS	PASS	FAIL	UNKNOWN
FAIL	FAIL	FAIL	FAIL
UNKNOWN	UNKNOWN	FAIL	UNKNOWN

OR works the other way round: one PASS is enough to pass, all FAILs give FAIL, and anything else is UNKNOWN. NOT swaps PASS and FAIL and leaves UNKNOWN as it is. At the end, PASS is shown as Eligible, FAIL as Not eligible, and UNKNOWN as Need more information, together with the list of missing answers. When one failed condition already settles a group, the engine drops the missing answers inside that group, so users are not asked questions that would not change anything.

8.2 Suggesting the next question

After a check, the engine looks at all the Need more information results and counts how often each missing answer comes up. It then suggests the most common one, because answering it will settle the largest number of schemes.

8.3 Turning scheme text into rules

Our extractor (api/ingest/extract.py) reads the Eligibility and Exclusions sections of each scheme and suggests a rule tree. Every suggested condition keeps the sentence it came from. If a sentence cannot be expressed using our list of allowed questions, it goes into unmapped_conditions so that the user can see it. Over the 570 schemes the extractor suggested 922 conditions, and 501 of the suggested rule trees passed our automatic checks. The validator (api/engine/validate.py) makes sure that only allowed questions and operators are used, that values match the question type, that ranges make sense, that each quoted sentence really exists in the scheme text, that the numbers in a rule also appear in its sentence, and that no two conditions contradict each other. Only rules that a person has checked are published: we checked 16 well-known schemes by hand. The other proposals are used only as "unverified" hints in the recommendations.

The validator also caught a mistake of our own. In our hand-written rules for the OBC post-matric scholarship we had typed an income limit of ₹2.5 lakh. The validator refused to publish it because the quoted sentence did not match the source, and when we checked, the actual limit was ₹1.5 lakh.

Question used in the rule	Conditions suggested
state	457
age	105
category	89
annual_income	76
is_citizen	37

disability_percent	33
is_bpl	28
gender	26
is_student	21
area_type	18

9. Grounded retrieval-augmented Q&A

- **Splitting the text.** Each scheme is cut along its own sections (Details, Benefits, Eligibility, Exclusions, Application, Documents, FAQ) and then into pieces of at most 350 words that overlap by one sentence. Before a piece is embedded we add the scheme name and section in front of it, so that it still makes sense on its own.
- **Finding the right pieces.** Keyword and similarity results are merged with Reciprocal Rank Fusion and the best six pieces are kept. Results can be limited to central schemes plus those of the user's state.
- **Sending each question to the right place.** "Am I eligible for APY?" goes to the rules engine. "Schemes for women entrepreneurs" first finds likely schemes and then checks each one with the engine. "How do I apply?" is answered from the text. Questions that have nothing to do with schemes get a polite refusal.
- **Staying close to the sources.** Each sentence of an answer must point to one of the numbered pieces it came from. The code checks every citation, and if an answer has no valid citation it is replaced with "I don't have reliable information on that. Please check myscheme.gov.in."
- **The model never decides eligibility.** Verdicts only ever come from the rules engine. If no API key is set, the answer is built directly from the best-matching sentences, so the feature works offline too.

9.1 Fixing wrong answers found in testing

While testing the app ourselves we found that some short questions got answers about the wrong scheme. For example "car loan" returned text about education loans, and "atal yojna" and "aadhar card" were refused as off-topic. We traced this to three causes and fixed each one.

- **The relevance check was too loose.** A question word counted as covered if it appeared anywhere in the results put together, so any passage containing "loan" passed for "car loan". Now each passage must contain the words that say what the question is about. For questions of one or two such words, all of them must appear; for longer questions, most of them.
- **Misspellings were treated as off-topic.** The topic filter only knew correctly spelled words. We added a small spelling map (yojna to yojana, aadhar to aadhaar, scholership to scholarship, and others) and simple plural matching (fishermen and fisherman count as the same word). A question is now also accepted if it matches the name of a scheme in our database.
- **Amounts were split in half.** The sentence splitter treated the full stop in "Rs 10.00 lakh" as the end of a sentence. It now ignores a full stop that is directly followed by a digit.

We also changed what the user sees when a question is short or has no match. A short topic such as "atal yojna" now returns a list of the matching schemes with a one-line description and a source link for each. When nothing in our data matches, the app says so plainly and lists schemes whose names share

a word with the question, instead of answering about something else. A question that starts with "How do I apply" is now treated as a request for the application steps rather than an eligibility check.

Question	Before the fix	After the fix
car loan	Sentences about education loans	"I couldn't find a scheme about car loan", plus related loan schemes
atal yojna	Refused as off-topic	List of the Atal schemes with descriptions and sources
aadhar card	Refused as off-topic	Document lists that mention the Aadhaar card, with sources
How do I apply for Aam Aadmi Bima Yojana?	An eligibility verdict	The application steps from the scheme page

10. Application integration and functionality

Method	Path	What it does	Who can call it
GET	/api/health	Checks the server, the database and the replica set	Anyone
GET	/api/attributes	Returns the list of profile questions for the form	Anyone
GET	/api/schemes	Lists, filters and searches schemes page by page	Anyone
GET	/api/schemes/{slug}	One scheme with its rules, versions and how recently it was verified	Anyone
POST	/api/eligibility/check	Checks a profile against all schemes and saves the audit records	Anyone
GET	/api/eligibility/runs/{run_id}	Results of an earlier check	Owner or admin
GET	/api/eligibility/history	The logged-in user's past checks	User
POST	/api/auth/register · /api/auth/login	Create a citizen account, or log in on the Citizen or Admin tab	Anyone
GET · PUT · DELETE	/api/profile	Read, save (with consent) or delete the profile and history	User
POST	/api/chat	Answers a question and streams the reply with sources	Anyone, rate-limited
POST	/api/recommendations	Ranked schemes for a profile, with the reasons for each rank	Anyone
GET · POST · PATCH · DELETE	/api/applications[/…]	Save schemes, set status and deadline, tick documents; reminders	User
POST	/api/admin/schemes/{slug}/retire	Retire a scheme without deleting it	Admin
GET	/api/admin/stats · /api/admin/coverage[/nearby]	Dashboard numbers; coverage	Admin

		gaps; nearby states	
POST	/api/grievances	File a complaint about a scheme, optionally linked to one of your checks	User
GET	/api/grievances · /api/grievances/{ticket}	Your complaints, or one complaint with its timeline	Owner or admin
POST	/api/grievances/{ticket}/comments	Add a comment, or reopen a resolved complaint	Owner or admin
GET	/api/admin/grievances	All complaints, filtered by status, with the filer's email	Admin
POST	/api/admin/grievances/{ticket}/status	Change status and priority, with a note to the citizen	Admin

The **public site** is one long page that scrolls smoothly between sections. **Check** shows the questions in four groups across the page, then the ranked recommendations and the results with a reason for each condition. **Ask** streams answers with source links. **Schemes** opens a page with the rule tree and its versions. **Grievances** lets a citizen file and follow a complaint. **My applications** holds saved schemes, their checklist and reminders, and **My account** shows past checks and a button to delete one's data. Login is a **separate page** with Citizen and Admin tabs. The **officers' dashboard** has overview numbers, the coverage-gap map and tables, and the grievance queue.

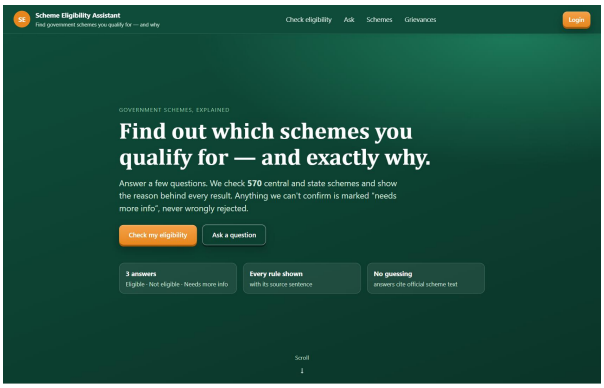


Figure 4 — Home page

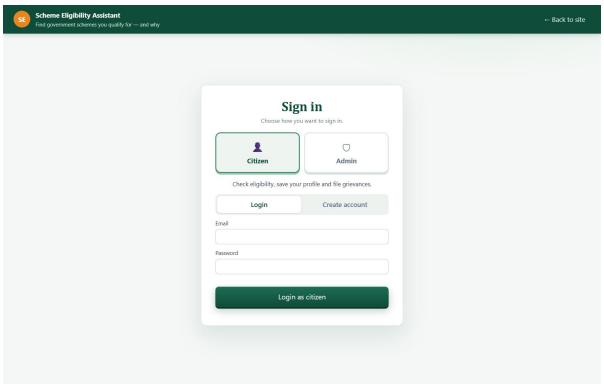


Figure 5 — Separate citizen and admin login

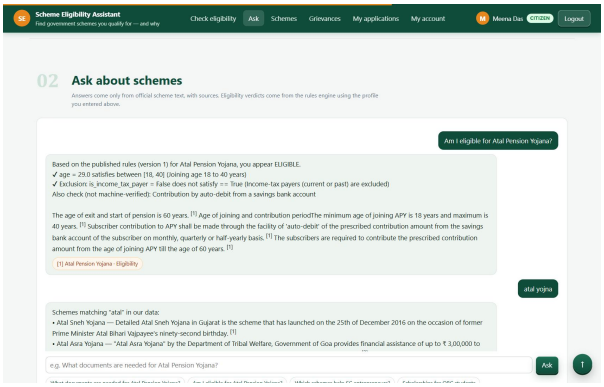


Figure 7 — Grounded answers

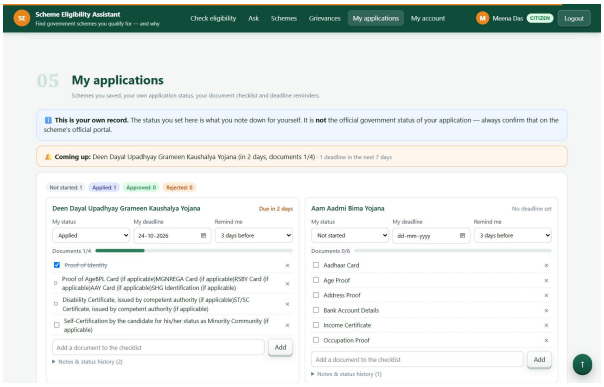


Figure 8 — Application tracker

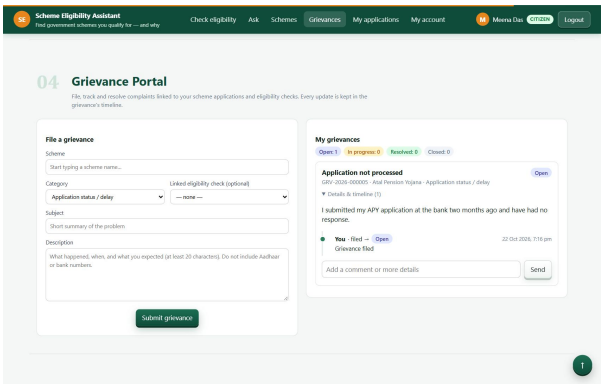


Figure 9 — Grievance portal

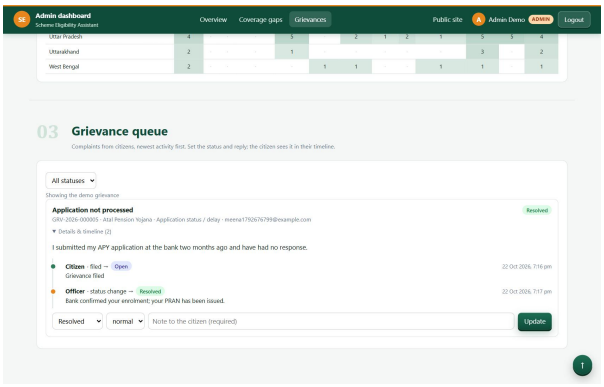


Figure 11 — Grievance queue admin

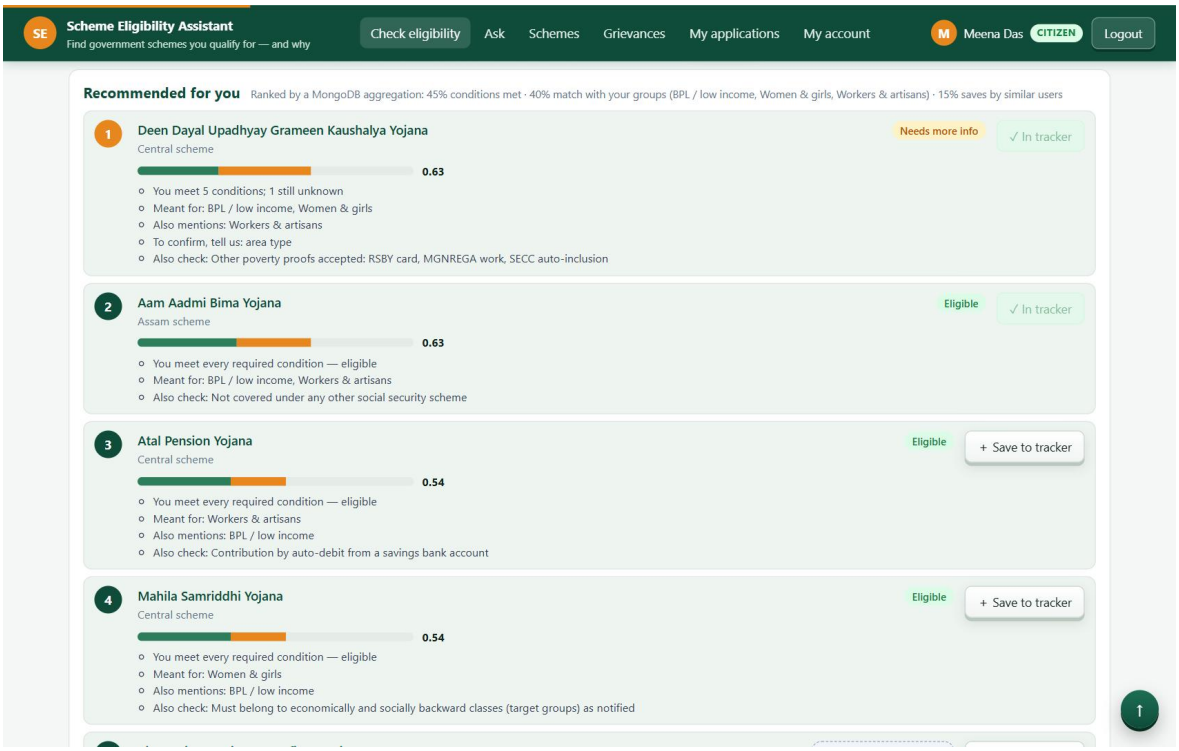


Figure 6 — Recommendations and results

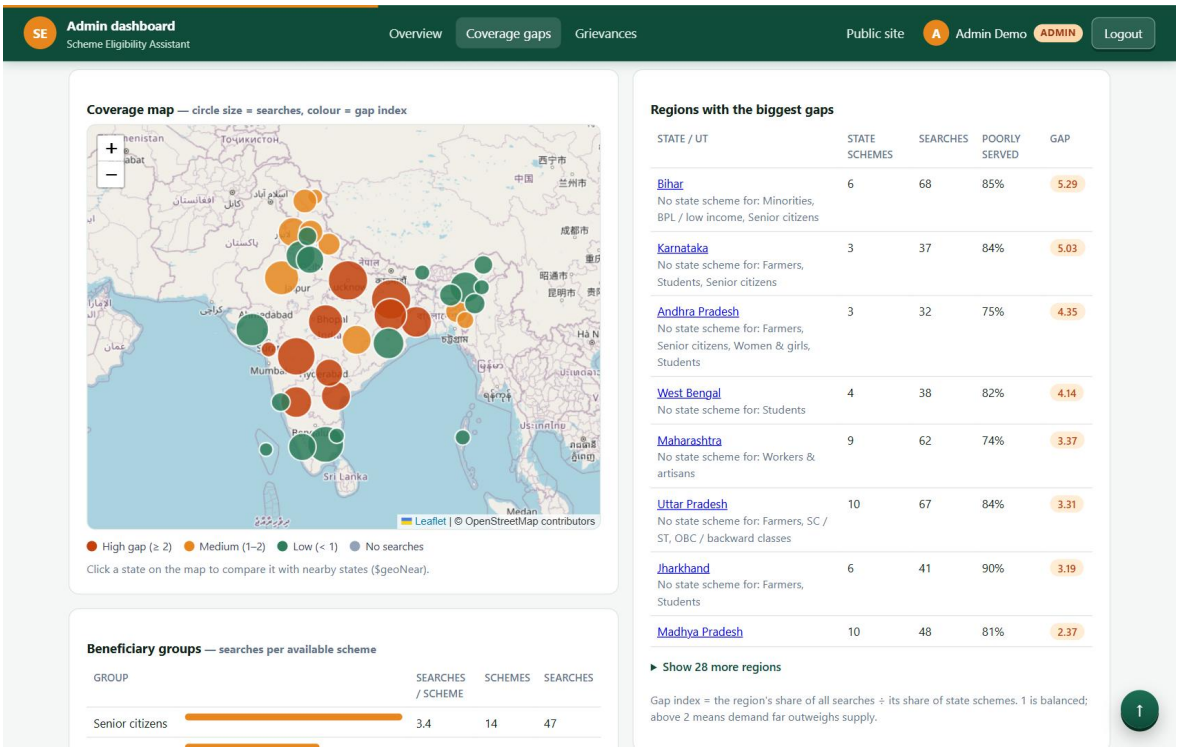


Figure 10 — Coverage gap analytics admin

11. Testing, security and data validation

11.1 Checking input at three levels

- **At the API**, Pydantic models enforce ranges (age 0–120, income zero or more, marks 0–100), allowed values taken from the question list, and length limits. A profile field we do not recognise is rejected.
- **Before publishing**, the rule validator from section 8.3 runs.
- **In the database**, strict JSON Schema validators and unique indexes act as the final safety net.

11.2 Login, roles and privacy

- Passwords are stored as PBKDF2-SHA256 hashes (240,000 rounds, a random salt per user) and compared in constant time.
- After login the user receives a JWT that expires after 12 hours; the signing secret is read from an environment variable.
- Anyone can check eligibility without an account, and their answers then stay in their own browser. Logged-in users can save a profile and see their history. Citizens and officers sign in on separate tabs; the server rejects a citizen account on the Admin tab, and only admins can open /api/admin/* (HTTP 403 otherwise).
- Audit records are only ever inserted by the API, and each one has a SHA-256 checksum. In a test we changed one stored value by hand and re-checking the record reported the checksum as invalid.
- We never ask for Aadhaar or any other ID number. A profile is saved only after the user agrees, the user can delete it together with their history at any time, and chat logs expire on their own.
- The chat endpoint is limited per IP address, using counters kept in a collection with a TTL index.

- A grievance can only be read or commented on by the user who filed it or by an admin; anyone else gets HTTP 403. Filing one requires login, a linked check must be from the user's own history, and the form asks users not to type Aadhaar or bank numbers.
- Queries are built as Python dictionaries rather than pasted strings, and user input is never used as a query operator, which protects against injection.

11.3 Automated tests

Test file	What it checks
tests/test_engine.py	AND, OR and NOT for every mix of PASS, FAIL and UNKNOWN; dropping questions that no longer matter; the suggested question
tests/test_properties.py	Hypothesis generates random rule trees and checks that NOT(NOT(x)) equals x and that answering a question never undoes a settled result
tests/test_fixtures.py	Sample profiles for the hand-checked schemes give the expected verdict
tests/test_validate.py	Unknown questions or operators, bad values, made-up quotes, numbers missing from quotes and contradictions are all rejected
tests/test_api.py	CRUD through HTTP on a test database; citizen vs admin login; checksum tamper check; validators; grievance flow; tracker (arrayFilters, $push / $pull, reminders); ranking (no ineligible schemes, audience check); coverage analytics and $geoNear

On the last run, **122 tests passed and 0 failed**, taking 16.7 seconds in total.

12. Results

Measure	Value
Scheme PDFs downloaded (duplicates removed)	601
Schemes parsed that have an Eligibility section	570
Central / state schemes	113 / 457
States and union territories covered	34
Conditions suggested by the extractor	922
Suggested rule trees that passed automatic checks	501 of 570
Schemes with hand-checked, published rules	16
Schemes published in total	17
Text pieces indexed for search	3991
Audit records in the database during benchmarking	6506
Automated tests	122 passed in 16.70s
Sample profiles that got the expected verdict	100% (35/35)
Median time to check all published schemes and save the records	9.8 ms

To test the search we used 200 questions where we already knew which scheme held the answer, and measured how often that scheme appeared in the top five results (Recall@5) and how high it was ranked (MRR).

Setup	Recall@5	MRR
vector · named	0.98	0.90
vector · descriptive	0.82	0.67
keyword · named	1.00	0.99
keyword · descriptive	0.15	0.11
hybrid · named	1.00	0.98
hybrid · descriptive	0.68	0.50

"Named" questions mention the scheme by name, while "descriptive" ones describe what the scheme does without naming it. Keyword search was almost perfect when the name was there (1.00) but poor otherwise (0.15); similarity search behaved the other way round. The combined search kept the strong result on named questions (1.00), but on descriptive questions it scored 0.68, which is lower than similarity search alone (0.82). Weak keyword matches pull the merged ranking down. We are reporting this as we found it; the fix we plan to try is to give the two lists different weights, or to use keyword search only when a scheme name is detected. Out of 10 questions that had nothing to do with schemes or had no answer in our data, 9 were correctly refused.

12.1 What the new features showed

We ran the recommendation ranking for four different people. Each gets a different list, led by schemes meant for their own groups; "(unverified)" marks schemes whose rules have not been hand-checked.

Person	Top three recommendations
Woman farmer, 45, Punjab, BPL	Deen Dayal Upadhyay Grameen Kaushalya Yojana; Mahila Samriddhi Yojana; Agri-Clinics And Agri-Business Centres Scheme
SC student, 19, Bihar	Credit Based Schemes For SC - Education Loan Scheme; Central Sector Scheme Of Scholarship For College And University Students; Credit Based Schemes For SC - Aajeevika Micro-Finance Yojana (Livelihood Microfinance Scheme) (unverified)
Person with disability, 24, Tamil Nadu	Deen Dayal Upadhyay Grameen Kaushalya Yojana; Agri-Clinics And Agri-Business Centres Scheme; Establishing Aavin Parlour for Differently Abled Persons (unverified)
Woman weaver, 29, Assam, BPL	Deen Dayal Upadhyay Grameen Kaushalya Yojana; Aam Aadmi Bima Yojana; Atal Pension Yojana

The coverage dashboard analysed 906 searches (900 of them simulated for the demo). The regions with the largest gap between searches and state schemes were:

State / UT	State schemes	Searches	Poorly served	Gap index
Bihar	6	68	85%	5.29
Karnataka	3	37	84%	5.03
Andhra Pradesh	3	32	75%	4.35
West Bengal	4	38	82%	4.14
Maharashtra	9	62	74%	3.37

13. Limitations and future work

- Our data is a snapshot of pages from 2024, so some details may be out of date. The app shows when each scheme was last verified and warns when that was more than 90 days ago. A real deployment should pull data from the official API Setu myScheme service.
- The extractor works with text patterns. A language model with a fixed output format could replace it while keeping the same validator and human review. Some conditions, such as "no family member in government service", still cannot be expressed and are shown as warnings.
- Our LSA embeddings only handle English well. A multilingual model would let someone ask in Hindi and still find the English text.
- The grievance portal and the tracker do not yet send email or SMS; users see updates when they log in.
- Recommendations from unreviewed schemes rely on machine-read rules and can miss details, which is why they carry an "unverified" label. Coverage analytics are by state, and the demo demand data is simulated.
- Next, we would like to move to MongoDB Atlas to use its built-in vector and text search indexes, add a Hindi interface, and build a test set that measures how faithful the answers are to their sources.

14. Conclusion

Working on this project showed us that MongoDB's document model suits government scheme data well. The data has many conditions and varies a lot from one scheme to another. Each scheme fits in one document along with its nested rules. Rule changes become new versions saved in a transaction, and every eligibility decision is a single record, with its own checksum, that can be re-checked later without any joins. Validators, unique and compound indexes, text and vector search, aggregation pipelines and TTL indexes together covered our needs for correct data, speed and privacy. On top of the database, the three-result rules engine gives answers that users can understand, the Q&A feature answers with sources but leaves the eligibility decision to the engine. Ranked recommendations, the application tracker, the grievance portal and the coverage-gap dashboard show how far aggregation pipelines, embedded arrays and geospatial indexes can carry an application.

15. References

1. MongoDB Manual: data modelling, schema validation, transactions, TTL indexes, text indexes and aggregation. https://www.mongodb.com/docs/manual/
2. MongoDB Atlas Vector Search and Atlas Search documentation. https://www.mongodb.com/docs/atlas/
3. myScheme, the national platform for government schemes. https://www.myscheme.gov.in/
4. shrijayan/gov_myscheme dataset on Hugging Face. https://huggingface.co/datasets/shrijayan/gov_myscheme
5. FastAPI documentation, https://fastapi.tiangolo.com/, and PyMongo documentation, https://pymongo.readthedocs.io/
6. Cormack, G. V., Clarke, C. L. A. and Büttcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. SIGIR.
7. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

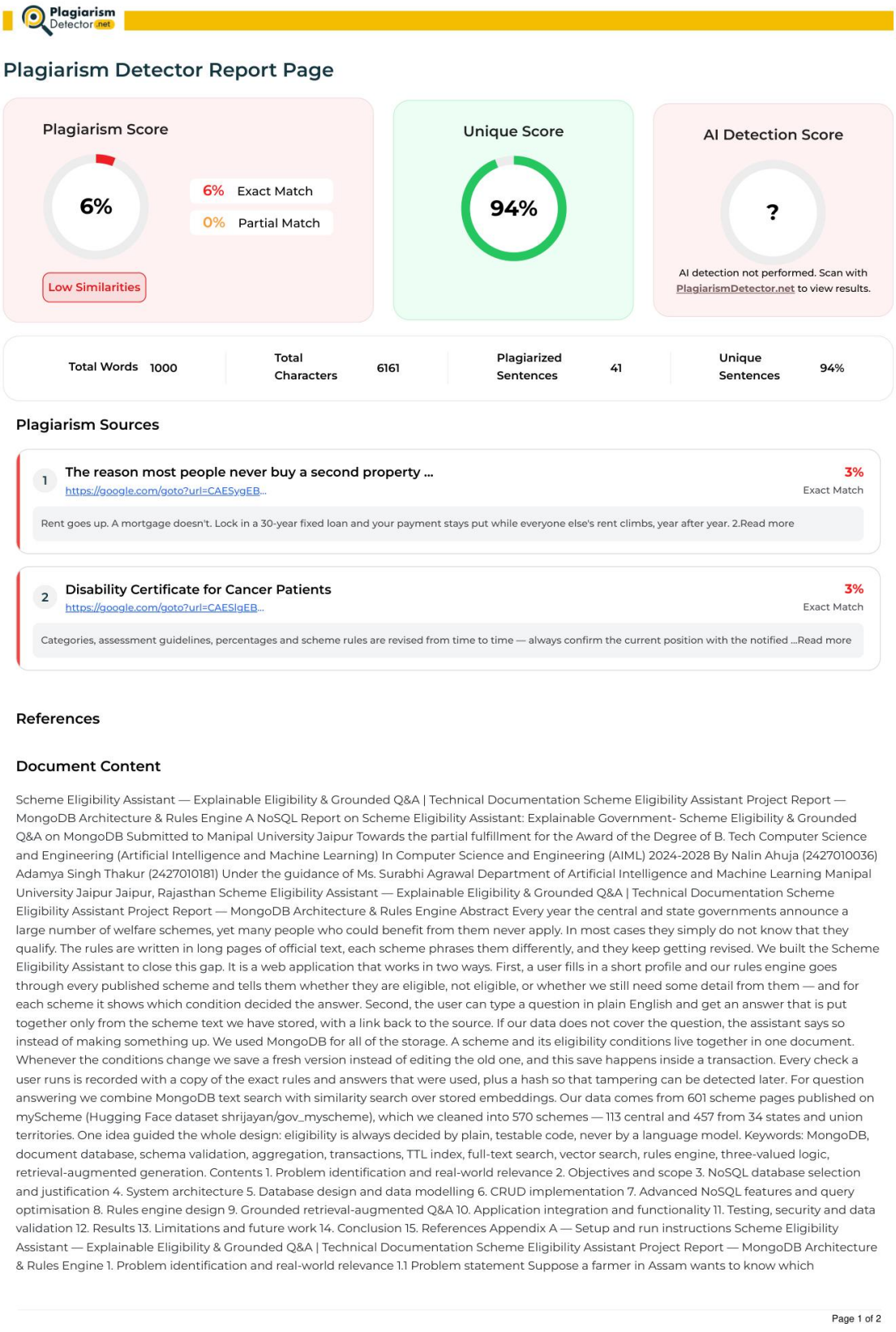
Appendix A — Setup and run instructions

```
pip install -r requirements.txt
powershell -ExecutionPolicy Bypass -File scripts\start_mongo.ps1
$env:HF_TOKEN="<your token>"; python scripts\download_hf.py
python scripts\seed.py
python scripts\simulate_searches.py
uvicorn api.main:app --port 8000
pytest -q
```

The commands, in order: install the Python packages; start MongoDB as a one-node replica set (needed for transactions); download the scheme PDFs; parse them, suggest and publish rules, and build the search index; add flagged demo searches for the coverage dashboard; start the app at http://127.0.0.1:8000 (API documentation at /docs); and run the tests. The seed script also creates two demo accounts, admin@example.org and user@example.org, with random passwords saved to the git-ignored file .env.demo for local use only.

Appendix B — Plagiarism report

The similarity check for this report is attached below, as generated by the plagiarism-checking tool.



Plagiarism Detector .net

Plagiarism Detector Report Page

Plagiarism Score

6%

6% Exact Match

0% Partial Match

Low Similarities

Unique Score

94%

AI Detection Score

?

AI detection not performed. Scan with PlagiarismDetector.net to view results.

Total Words	1000	Total Characters	6161	Plagiarized Sentences	41	Unique Sentences	94%

Plagiarism Sources

1 **The reason most people never buy a second property ...** 3%
https://google.com/goto?url=CAESygEB... Exact Match

Rent goes up. A mortgage doesn't. Lock in a 30-year fixed loan and your payment stays put while everyone else's rent climbs, year after year. 2.Read more

2 **Disability Certificate for Cancer Patients** 3%
https://google.com/goto?url=CAESlgEB... Exact Match

Categories, assessment guidelines, percentages and scheme rules are revised from time to time — always confirm the current position with the notified ...Read more

References

Document Content

Scheme Eligibility Assistant — Explainable Eligibility & Grounded Q&A | Technical Documentation Scheme Eligibility Assistant Project Report — MongoDB Architecture & Rules Engine A NoSQL Report on Scheme Eligibility Assistant: Explainable Government- Scheme Eligibility & Grounded Q&A on MongoDB Submitted to Manipal University Jaipur Towards the partial fulfillment for the Award of the Degree of B. Tech Computer Science and Engineering (Artificial Intelligence and Machine Learning) In Computer Science and Engineering (AIML) 2024-2028 By Nalin Ahuja (2427010036) Adamya Singh Thakur (2427010181) Under the guidance of Ms. Surabhi Agrawal Department of Artificial Intelligence and Machine Learning Manipal University Jaipur Jaipur, Rajasthan Scheme Eligibility Assistant — Explainable Eligibility & Grounded Q&A | Technical Documentation Scheme Eligibility Assistant Project Report — MongoDB Architecture & Rules Engine Abstract Every year the central and state governments announce a large number of welfare schemes, yet many people who could benefit from them never apply. In most cases they simply do not know that they qualify. The rules are written in long pages of official text, each scheme phrases them differently, and they keep getting revised. We built the Scheme Eligibility Assistant to close this gap. It is a web application that works in two ways. First, a user fills in a short profile and our rules engine goes through every published scheme and tells them whether they are eligible, not eligible, or whether we still need some detail from them — and for each scheme it shows which condition decided the answer. Second, the user can type a question in plain English and get an answer that is put together only from the scheme text we have stored, with a link back to the source. If our data does not cover the question, the assistant says so instead of making something up. We used MongoDB for all of the storage. A scheme and its eligibility conditions live together in one document. Whenever the conditions change we save a fresh version instead of editing the old one, and this save happens inside a transaction. Every check a user runs is recorded with a copy of the exact rules and answers that were used, plus a hash so that tampering can be detected later. For question answering we combine MongoDB text search with similarity search over stored embeddings. Our data comes from 601 scheme pages published on myScheme (Hugging Face dataset shrijayan/gov_myscheme), which we cleaned into 570 schemes — 113 central and 457 from 34 states and union territories. One idea guided the whole design: eligibility is always decided by plain, testable code, never by a language model. Keywords: MongoDB, document database, schema validation, aggregation, transactions, TTL index, full-text search, vector search, rules engine, three-valued logic, retrieval-augmented generation. Contents 1. Problem identification and real-world relevance 2. Objectives and scope 3. NoSQL database selection and justification 4. System architecture 5. Database design and data modelling 6. CRUD implementation 7. Advanced NoSQL features and query optimisation 8. Rules engine design 9. Grounded retrieval-augmented Q&A 10. Application integration and functionality 11. Testing, security and data validation 12. Results 13. Limitations and future work 14. Conclusion 15. References Appendix A — Setup and run instructions Scheme Eligibility Assistant — Explainable Eligibility & Grounded Q&A | Technical Documentation Scheme Eligibility Assistant Project Report — MongoDB Architecture & Rules Engine 1. Problem identification and real-world relevance 1.1 Problem statement Suppose a farmer in Assam wants to know which

Page 1 of 2

government schemes she can apply for. Today she would have to open scheme pages one by one and work out conditions such as age limits, income ceilings, caste category, land owned, BPL status and state of residence — and then do the same for the next scheme, and the next. Most people do not have the time for this, so they either give up or pay an agent. Staff at Common Service Centres and block offices run into the same problem, only with many more applicants. A general chatbot is not a good answer either: if it confidently tells someone the wrong thing about a benefit, that person can lose money they were entitled to. 1.2 Who it serves · Citizens, who want to find out what they can claim without handing over identity documents. · CSC operators and block officers, who screen many applicants a day and need answers they can explain. · Scheme administrators, who need to change rules safely and look back at decisions made earlier. 1.3 Why it matters · A lot of welfare money goes unclaimed because the people it is meant for never hear that they are eligible. · A useful answer has to be explainable — the person should see which condition they met or missed — and reproducible, meaning we can show what the rules were on the day they checked. · Scheme rules are revised from time to time. Showing when our data was last verified, and keeping old rule versions instead of overwriting them, are practical requirements for a system like this. 2. Objectives and scope 2.1 Objectives · Store scheme data whose structure varies (central and state schemes are not laid out the same way) in a document database. · Write eligibility conditions as data — a tree of rules — so that a single piece of code can check all the schemes, instead of writing separate code for each one. · Give one of three answers for every scheme (Eligible, Not eligible, Need more information), explain each condition, and suggest the one question that would help the user most. · Keep a record of every check that cannot be quietly edited and that can be re-run later from what it stored. · Never edit published rules in place; publish each change as a new version in a single atomic step. · Answer free-text questions using only stored scheme text, show the sources, and refuse when the sources do not contain the answer. · Cover the database topics of the course: CRUD, indexing, aggregation, validation, TTL, text and vector search, login and roles,